

Datamanagement & Analytics

Prof. Dr. André Drews

Fachbereich Maschinenbau und Wirtschaft

04. Mai 2026

- nur für den Gebrauch innerhalb des Kurses -

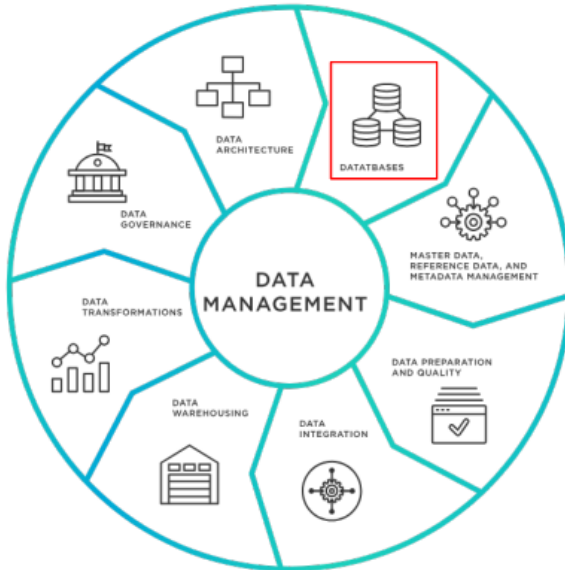
Vorlesungskapitel 4

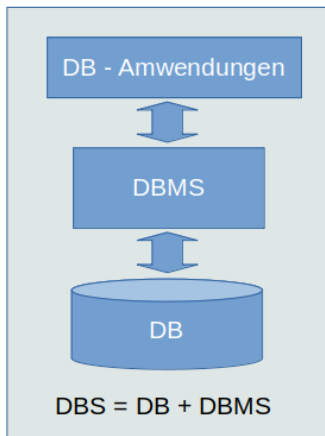
- SQL/No-SQL Datenbanken, Anwendung von Datenbanken in der Logistik -

Lernziele

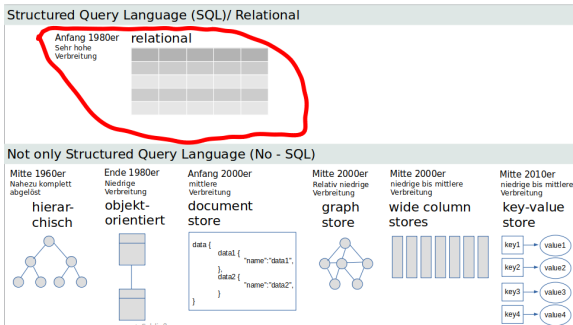
- Kompetenz im Bereich Datenbanken (SQL, No-SQL)
- Anwendungskompetenz im Bereich Einsatz von Datenbanken in der Logistik

1. SQL-Datenbank
2. No-SQL Datenbanken (GDBMS, IMDB, ZRDBMS)
3. Beispielhafte Anwendung verschiedener Datenbankarten in der Logistik

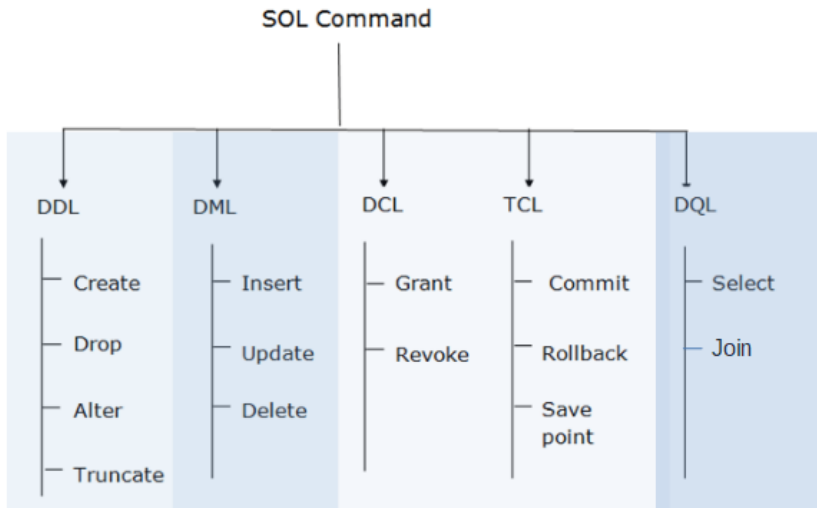




- DB-Anwendungen: Alle Arten von Anwendungen (User Interface, Domänen-Anwendungen)
- Datenbankmanagementsystem (DBMS): Sammlung von ausführbaren Programmen, welche zur Umsetzung aller Zugriffe auf eine Datenbank eingesetzt werden
- Datenbank (DB): Sammlung strukturierter und dauerhaft gespeicherter Fakten für eine oder viele Domänenanwendungen
- Datenbanksystem (DS): Durch ein DBMS zugreifbare DB für eine oder viele Domänenanwendungen



SQL Sublanguages



SQL-Befehl CREATE

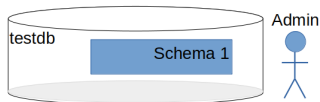
Der SQL-Befehl CREATE dient dazu, folgende Komponenten anzulegen:

- Datenbank
- Schema
- Datenbank-User
- Tabelle
- Index

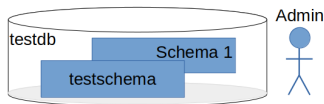
Ausdruck:	Bedeutung:
<code>CREATE DATABASE testdb;</code>	Erzeugt ein Datenbankschema
<code>CREATE SCHEMA testschema;</code>	Erzeugt ein Datenbankschema
<code>CREATE USER testuser;</code>	Erzeugt einen Datenbank User
<code>CREATE TABLE testtable (test_id INTEGER);</code>	Erzeugt eine Tabelle erzeugen
<code>CREATE INDEX idx_test ON testtable(name);</code>	Erzeugt Indizes auf einer Tabelle (für performante Suchabfragen)

Data Definition Language (DDL) - CREATE

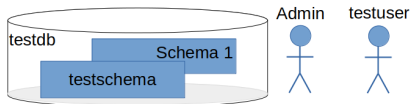
```
CREATE DATABASE testdb;
```



```
CREATE SCHEMA testschema;
```

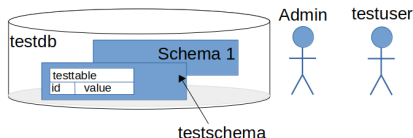


```
CREATE USER testuser;
```

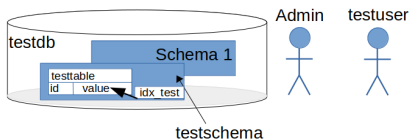


Data Definition Language (DDL) - CREATE

```
CREATE TABLE testtable ( test_id INTEGER);
```



```
CREATE INDEX idx_test ON testtable(name);
```



SQL-Befehl DROP

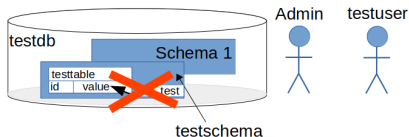
Der SQL-Befehl DROP dient dazu, folgende Komponenten aus dem DBMS zu entfernen:

- Datenbank
- Schema
- Datenbank-User
- Tabelle
- Index

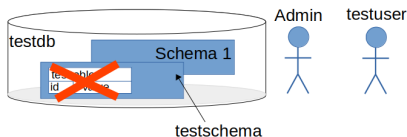
Ausdruck:	Bedeutung:
<code>DROP DATABASE testdb;</code>	Entfernt ein Datenbankschema
<code>DROP SCHEMA testschema;</code>	Entfernt ein Datenbankschema
<code>DROP USER testuser;</code>	Entfernt einen Datenbank User
<code>DROP TABLE testtable;</code>	Entfernt eine Tabelle erzeugen
<code>DROP INDEX idx_test;</code>	Entfernt Indizes auf einer Tabelle (für performante Suchabfragen)

Data Definition Language (DDL) - DROP

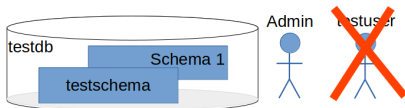
```
DROP INDEX idx_test;
```



```
DROP TABLE testtable;
```

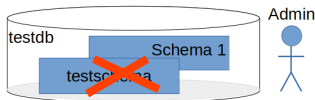


```
DROP SCHEMA testuser;
```

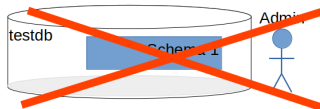


Data Definition Language (DDL) - DROP

```
DROP SCHEMA testschema;
```



```
DROP DATABASE testdb;
```



SQL-Befehl ALTER

Der Befehl ALTER ändert das Schema: Es können z.B. Spalten einer Tabelle hinzugefügt oder gelöscht werden (ALTER TABLE). Tabellen oder Spalten können umbenannt werden. Der Datentyp einer Spalte kann verändert werden (ALTER COLUMN).

Ausdruck:	Bedeutung:
ALTER TABLE testtable ADD COLUMN description VARCHAR;	Spalte hinzufügen
ALTER TABLE testtable DELETE COLUMN description;	Löscht Spalte description
ALTER TABLE testtable ALTER COLUMN description TYPE text;	Spaltentyp änderSpaltentyp ändern
ALTER TABLE testtable RENAME COLUMN description TO descr;	Benennt Spalte um
ALTER TABLE testtable RENAME TO test;	Benennt Tabelle um

Data Definition Language (DDL) - ALTER

- `ALTER TABLE testtable ADD COLUMN description VARCHAR;`
- `ALTER TABLE testtable DELETE COLUMN description;`

testtable		→	testtable		
id	value		id	value	description
INTEGER	INTEGER		INTEGER	INTEGER	VARCHAR

- `ALTER TABLE testtable ALTER COLUMN description TYPE text;`
- `ALTER TABLE testtable RENAME COLUMN description TO descr;`
- `ALTER TABLE testtable RENAME TO test;`

test		
id	value	descr
INTEGER	INTEGER	text

SQL-Befehl TRUNCATE

- TRUNCATE löscht den Inhalt einer Tabelle.
- TRUNCATE gehört zur DDL, da es die Tabelle zunächst löscht (DROP) und danach wieder mit allen Metadaten neu erzeugt.

Ausdruck:	Bedeutung:
TRUNCATE TABLE testtable;	Löscht den Inhalt der Tabelle

test		
id	value	descr
INT	INT	VARCHAR
1	2	"abc"
2	243	"def"
3	7	"ghi"
4	10	"ijk"
5	12	"lmn"



test		
id	value	descr
INT	INT	VARCHAR

SQL-Befehl DELETE

- DELETE löscht den Inhalt einer Tabelle.
- Anders als bei TRUNCATE, bleibt die Tabelle erhalten und wird Zeile für Zeile gelöscht. Deshalb gehört DELETE zur DML
- DELETE ist langsamer als TRUNCATE.

Ausdruck:	Bedeutung:
DELETE * FROM testtable (optional WHERE ...);	Löscht den Inhalt der Tabelle

test		
id	value	descr
INT	INT	VARCHAR
1	2	"abc"
2	243	"def"
3	7	"ghi"



test		
id	value	descr
INT	INT	VARCHAR

- nur für den Gebrauch innerhalb der Vorlesung -

SQL-Befehl INSERT

INSERT erzeugt eine oder mehrere Zeilen mit Inhalt in einer Tabelle.

Ausdruck:	Bedeutung:
<code>INSERT INTO testdb VALUES (1,2,'test');</code>	Importiert eine Zeile
<code>INSERT INTO testdb (description) VALUE ('abc')</code>	Importiert nur einen Feldinhalt
<code>INSERT INTO testdb VALUES (DEFAULT, 2, 'abc');</code>	Importiert eine Zeile, wobei das erste Feld vom Typ SERIAL ist (autoincrement)
<code>INSERT INTO testdb values (1,2,'abc'), (2,3,'def');</code>	Importiert mehrere Zeilen

Data Manipulation Language (DDL) - INSERT

- INSERT INTO testdb(description) VALUES (1,2,'test'), (2,4,'abc2');

test		
id	value	descr
INT	INT	VARCHAR

→

test		
id	value	descr
INT	INT	VARCHAR
1	2	test
2	4	abc2

SQL-Befehl UPDATE

UPDATE aktualisiert eins oder mehrere Felder einer oder mehrerer Spalten.

Ausdruck:	Bedeutung:
<code>UPDATE testtable set descr='123' WHERE descr='test';</code>	Updated Spalte descr mit dem Wert '123'
<code>UPDATE employees SET sales_count = sales_count + 1 WHERE id = (SELECT sales_person FROM accounts WHERE name = 'Acme Corporation');</code>	Erhöht sales_count in Tabelle employees um 1 für Verkäufer der Firma 'Acme Corporation' (von Tabelle accounts)

Data Manipulation Language (DDL) - UPDATE

- `UPDATE testtable set descr='123' WHERE descr='test';`

testtable		
id	value	descr
INT	INT	VARCHAR
1	2	test
2	4	abc2

 →

testtable		
id	value	descr
INT	INT	VARCHAR
1	2	123
2	4	abc2

SQL-Befehle GRANT, REVOKE

GRANT erteilt Zugangsgriffsberechtigungen für Rollen/User auf Datenbankobjekte wie

- Datenbanken
- Views
- Tabellen

Ausdruck:	Bedeutung:
<code>GRANT INSERT on testtable TO PUBLIC;</code>	Jeder User darf in testtable Daten einfügen
<code>GRANT ALL PRIVILEGES ON testdb TO user1;</code>	User user1 hat admin Rechte auf Datenbank testdb

Auswahl an möglichen Berechtigungen:

- SELECT: erlaubt das Auslesen aller oder spezieller Spalten einer Tabelle oder View.
- INSERT: erlaubt das Anlegen einer neuen Zeilen.
- UPDATE: erlaubt ein Update für eine speziellen Tabelle.
- DELETE: erlaubt das Löschen auf einer speziellen Tabelle.
- TRUNCATE: erlaubt eine spezielle Tabelle zu löschen.
- REFERENCES: erlaubt das Anlegen von Foreign Key Beziehungen.
- CREATE: erlaubt das Anlegen eines neuen Schemas für eine Datenbank.
- CONNECT: user darf sich mit Datenbank verbinden.
- TEMPORARY TABLE: erlaubt das Anlegen von Temporary Tables.
- ALL PRIVILEGES: alle Privilegien werden erteilt

SQL-Befehl REVOKE

REVOKE nimmt Zugangsgriffsberechtigungen für Rollen/User auf Datenbankobjekte wie

- Datenbanken
- Views
- Tabellen

zurück.

Ausdruck:	Bedeutung:
<code>REVOKE privileges ON object FROM user;</code>	privileges sind die GRANT Privileges, objects sind Datenbankobjekte
<code>REVOKE SELECT ON products FROM PUBLIC;</code>	Alle User dürfen kein SELECT mehr auf Tabelle products absetzen.

Transaction Controll Language (TCL) - COMMIT, ROLLBACK, SAVEPOINT

SQL-Befehle COMMIT, ROLLBACK, STOPPOINT

Transaktionen sind Folgen von SQL Anweisungen. Im Misserfolgsfall kann der Zustand der Datenbank auf einen definierten, stabilen Punkt zurückgesetzt werden. Transaktionen werden über COMMIT, ROLLBACK und SAVEPOINT definiert.

```
-- begin with transaction
BEGIN;
-- save point setzen
SAVEPOINT sp1;
INSERT INTO wines VALUES('Chateau Lafite 2003', '24');
ROLLBACK TO sp1;
UPDATE wines SET stock = stock + 24 WHERE winename = 'Chateau
Lafite 2003';
-- continue with other operations, and eventually
COMMIT;
```

Beispiel: Data Query Language (DQL) - SELECT

SQL-Befehl SELECT

SELECT dient dazu Tabelleninhalte zurückzugeben.

Ausdruck:	Bedeutung:
<code>SELECT * FROM testtable;</code>	Gibt alle Inhalte der Tabelle zurück
<code>SELECT * FROM testtable, testtable2;</code>	Gibt das Kreuzprodukt von testtable und testtable2 zurück
<code>SELECT description FROM testtable, testtable2;</code>	Gibt den Inhalt der Spalte description von testtable zurück
<code>SELECT id, description FROM testtable, testtable2;</code>	Gibt die Inhalte der Spalten id und description von testtable zurück.

Beispiel: Data Query Language (DQL) - SELECT

- SELECT * FROM KONTO, PRODUKT;

```
webshop=> select * from produkt;
```

1		2,12 €
6		2,30 €
7		3,40 €
8		1,40 €

```
webshop=> select * from konto;
```

1		0,00 €		DE2132141241
2		0,00 €		DE2132141241

```
webshop=> select * from konto,produkt  
;
```

1		0,00 €		DE2132141241		1		2,12 €
1		0,00 €		DE2132141241		6		2,30 €
1		0,00 €		DE2132141241		7		3,40 €
1		0,00 €		DE2132141241		8		1,40 €
2		0,00 €		DE2132141241		1		2,12 €
2		0,00 €		DE2132141241		6		2,30 €
2		0,00 €		DE2132141241		7		3,40 €
2		0,00 €		DE2132141241		8		1,40 €

Beispiel: Data Query Language (DQL) - SELECT

Ausdruck:	Bedeutung:
<code>SELECT * FROM testtable WHERE description='test';</code>	Die Zeilen von testtable werden zurückgegeben, bei denen Spalte description den Wert test annimmt.
<code>SELECT 3*7 AS RESULT;</code>	RESULT ist Alias und ergibt 21
<code>SELECT DISTINCT column1 FROM testtable;</code>	gibt den eindeutigen Inhalt (keine gleichen Feldinhalte) der Spalte column1 zurück
<code>SELECT column1,column2 FROM testtable ORDER BY column1 ASC;</code>	die Inhalte von column1 und column2 werden nach der Reihenfolge von column1 geordnet (ASC: aufsteigend, DSC: absteigend)
<code>SELECT column1,column2 FROM testtable GROUP BY column1;</code>	die Inhalte von column1 und column2 werden nach der nach Gleichheit von Spalte column2 gruppiert

Beispiel: Data Query Language (DQL) - SELECT

Aggregation mit select:

Ausdruck:	Bedeutung:
<code>SELECT MAX(umsatz) FROM verkauf;</code>	den Maximalwert des Umsatzes bestimmen
<code>SELECT AVG(alter) FROM kunde;</code>	das Arithmetische Mittel des Alters bestimmen
<code>SELECT SUM(transaktionen) FROM verkauf;</code>	die Summe der Transaktionen bestimmen

SQL-Befehl JOIN

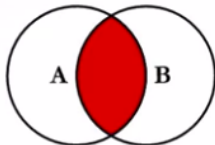
JOIN verbindet zwei Tabellen miteinander. Es gibt

- INNER JOIN: Schnittmenge
- LEFT JOIN: Linker Teil der zu verbindenen Mengen/Relationen
- RIGHT JOIN: Rechter Teil der zu verbindenen Mengen/Relationen
- FULL OUTER JOIN: Rechter und linker Teil der zu verbindenen Mengen/Relationen

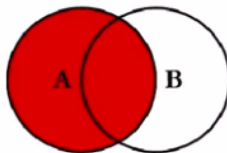
Ausdruck: z.B.

```
SELECT * FROM weather INNER JOIN  
        cities ON (weather.city= citites.name);
```

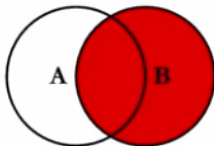
Data Query Language (DCL) - JOIN



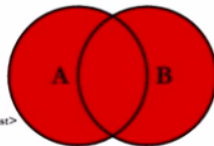
```
SELECT <select_list>  
FROM TableA A  
INNER JOIN TableB B  
ON A.Key = B.Key
```



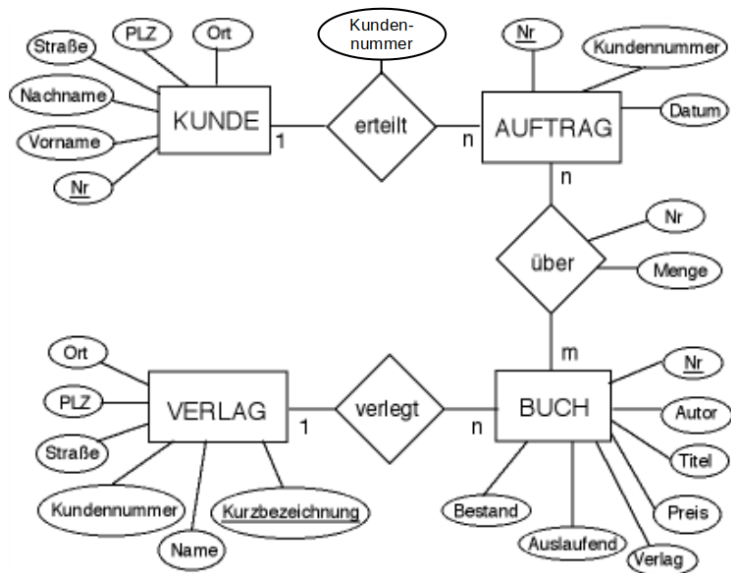
```
SELECT <select_list>  
FROM TableA A  
LEFT JOIN TableB B  
ON A.Key = B.Key
```



```
SELECT <select_list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.Key = B.Key
```



```
SELECT <select_list>  
FROM TableA A  
FULL OUTER JOIN TableB B  
ON A.Key = B.Key
```



1. Normalform - Atomistizität

Studenten

Vorname	Nachname	Informatikkenntnisse
Thomas	Müller	Java, C++, PHP
Ursula	Meier	PHP, Java
Igor	Müller	C++, Java

Ausgangslage

Resultat nach Normalisierung



Studenten

Vorname	Nachname	Informatikkenntnisse
Thomas	Müller	C++
Thomas	Müller	PHP
Thomas	Müller	Java
Ursula	Meier	Java
Ursula	Meier	PHP
Igor	Müller	Java
Igor	Müller	C++

2. Normalform - Funktionale Trennung

Studenten

IDSt	Nachname	IDProf	Professor	Note
1	Müller	3	Schmid	5
2	Meier	2	Borner	4
3	Tobler	1	Bernasconi	6

Ausgangslage



Resultat nach Normalisierung

Studenten

ID	Nachname
1	Müller
2	Meier
3	Tobler

Professoren

IDProf	Professor
1	Bernasconi
2	Borner
3	Schmid

Noten

IDST	IDProf	Note
1	3	5
2	2	4
3	1	6

3. Normalform - Vermeidung von Transzitivität

Lieferant

ID	Name	Konto_Nr	Bank_Clearing_Nr	Bank
----	------	----------	------------------	------



Ausgangslage

Resultat nach Normalisierung

Lieferant

ID	Name	Konto_Nr	Bank_Clearing_Nr
----	------	----------	------------------

Bank

Bank_Clearing_Nr	Bank
------------------	------

Quelle: http://www.gitta.info/LogicModelin/de/html/DataConsiten_Norm3NF.html

1. SQL-Datenbanken
2. No-SQL Datenbanken
 - 2.1 Dokumentenbasierte Datenbanken
 - 2.2 Graphen Datenbanken
 - 2.3 Key Value Datenbanken
 - 2.3 Zeitreihen Datenbanken
3. Beispielhafte Anwendung verschiedener Datenbankarten in der Logistik

SQL vs. No-SQL

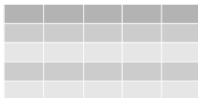
Relationale Datenbank	No-SQL Datenbank
Unterstützt eine mächtige Query/Abfrage-Sprache (SQL)	Unterstützt eine sehr einfache Query/Abfrage-Sprache (z.B. CQL)
Besitzt ein festes Schema	Besitzt kein festes Schema
Folgt ACID (Atomicity, Consistency, Isolation, Durability)	Ist nur "eventuell" konsistent
Unterstützt Transaktionen	Unterstützt keine Transaktionen
Skaliert schlecht bei großen Datenmengen	skaliert gut bei großen Datenmengen
Manche Operationen (z.B. joins) sind langsam	Operationen sind performant

Dokumentbasierte Datenbank: Anwender Mongo-DB

Structured Query Language (SQL)/ Relational

Anfang 1980er
Sehr hohe
Verbreitung

relational



No not only Structured Query Language (No - SQL)

Mitte 1960er
Nahezu komplett
abgelöst

hierar-
chisch



Ende 1980er
Niedrige
Verbreitung
objekt-
orientiert



Anfang 2000er
mittlere
Verbreitung
document
store

```
data {  
  data1 {  
    "name": "data1",  
  },  
  data2 {  
    "name": "data2",  
  }  
}
```

Mitte 2000er
Relativ niedrige
Verbreitung

graph
store



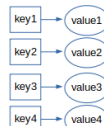
Mitte 2000er
niedrige bis mittlere
Verbreitung

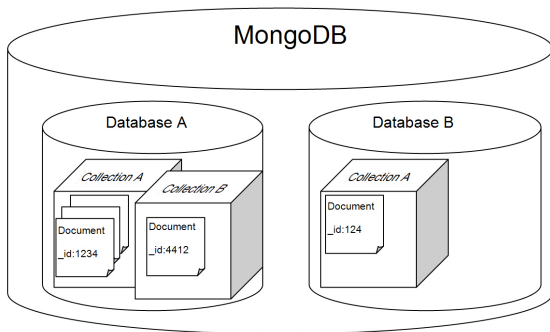
wide column
stores



Mitte 2010er
niedrige bis mittlere
Verbreitung

key-value
store





- Daten werden in json-Dokumenten gespeichert (RDBMS-pendant: Tabellen-Zeile).
- json-Dokumente werden zu collections zusammengefasst (RDBMS-pendant: Tabelle).
- Eine Menge von collection entspricht einer Datenbank.

- **Dokumentenbasierte Datenbanken (DBMS):**

NoSQL-Datenbanken wie MongoDB werden in der Logistik eingesetzt, wenn flexiblere Datenmodelle oder eine skalierbarere Architektur benötigt werden. Sie können für Anwendungen verwendet werden, die unstrukturierte oder halbstrukturierte Daten speichern, wie z.B. Protokolldaten, Sensordaten und andere nicht-relationale Daten.

```
{
  "timeStamp": "2012-06-24 T 10:45 UTC",
  "user": "John"
  "samplingRate": "60 sec"
  "totalNumberOfPlots": "25180"
  "geographicalLocation":
    [.....
      {
        "longitude": "149.120575"
        "latitude": "-35.275291"
        "plotNumber": "12458"
        "airTemperature": "5.1C"
        "airHumidity": "20%"
        "airStress": "low"
        "phytophthoraDisease": "not-infected"
      }
      .....
    ]
}
```

Parameters and data values are only used for demonstration purposes. The real data stream would contain many data fields

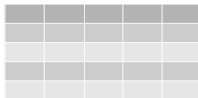
Beispiel W3Schools: Mongo TB - Python Treiber pymongo DB

Link zum w3schools tutorial

Structured Query Language (SQL)/ Relational

Anfang 1980er
Sehr hohe
Verbreitung

relational



No not only Structured Query Language (No - SQL)

Mitte 1960er
Nahezu komplett
abgelöst

hierar-
chisch



Ende 1980er
Niedrige
Verbreitung

objekt-
orientiert



Anfang 2000er
mittlere
Verbreitung

document
store

```
data {  
  data1 {  
    "name": "data1",  
  },  
  data2 {  
    "name": "data2",  
  }  
}
```

Mitte 2000er
Relativ niedrige
Verbreitung

graph
store



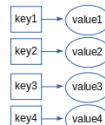
Mitte 2000er
niedrige bis mittlere
Verbreitung

wide column
stores



Mitte 2010er
niedrige bis mittlere
Verbreitung

key-value
store



* Schließen

Graphenbasierte Datenbank: Neo4j

← → ↻ https://console.neo4j.org

Apps SQL TryIt Edit... The Data Visu... rgraph Outlier Detect... PostgreSQL... 2business pro... LKW-Telemati...

Clear DB Help Share Toggle Viz Options

Graph Setup:

```
create (Neo:Crew {name:'Neo'}), (Morpheus:Crew {name: 'Morpheus'}), (Trinity:Crew {name: 'Trinity'}), (Cypher:Crew:Matrix {name: 'Cypher'}), (Smith:Matrix {name: 'Agent Smith'}), (Architect:Matrix {name:'The Architect'}),
(Neo)-[:KNOWS]->(Morpheus), (Neo)-[:LOVES]->(Trinity), (Morpheus)-[:KNOWS]->(Trinity),
(Morpheus)-[:KNOWS]->(Cypher), (Cypher)-[:KNOWS]->(Smith), (Smith)-[:CODED_BY]->(Architect)
```

Query:

```
match (n:Crew)-[:KNOWS*]-(m) where n.name='Neo' return n as Neo,r,m
```

Neo	r	m
(0:Crew {name:"Neo"})	[(0)-[:KNOWS]->(1)]	(1:Crew {name:"Morpheus"})
(0:Crew {name:"Neo"})	[(0)-[:KNOWS]->(1), (1)-[:KNOWS]->(2)]	(2:Crew {name:"Trinity"})
(0:Crew {name:"Neo"})	[(0)-[:KNOWS]->(1), (1)-[:KNOWS]->(3)]	(3:Crew:Matrix {name:"Cypher"})
(0:Crew {name:"Neo"})	[(0)-[:KNOWS]->(1), (1)-[:KNOWS]->(3), (3)-[:KNOWS]->(4)]	(4:Matrix {name:"Agent Smith"})

Query took 86 ms and returned 4 rows. [Result Details](#)

You can modify and query this graph by entering statements in the input field at the bottom.
For some syntax help hit the [Help](#) button.
If you want to share your graph, just do it with [Share](#)

```
graph TD
    Neo((Neo)) -- KNOWS --> Morpheus((Morpheus))
    Neo -- LOVES --> Trinity((Trinity))
    Morpheus -- KNOWS --> Trinity
    Morpheus -- KNOWS --> Cypher((Cypher))
    Cypher -- KNOWS --> Smith((Agent Smith))
    Smith -- CODED_BY --> Architect((The Architect))
```

```
match (n:Crew)-[:KNOWS*]-(m) where n.name='Neo' return n as Neo,r,m
```

Beispiel Analyse der Abhängigkeiten der Supply Chain für die Automobil Industrie supply chain



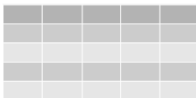
<https://neo4j.com/press-releases/neo4j-for-supply-chain-analytics/>

- **Graphdatenbanken:** Graphdatenbanken wie Neo4j werden in der Logistik eingesetzt, wenn es darum geht, komplexe Beziehungen und Netzwerke zwischen verschiedenen Entitäten zu modellieren und zu analysieren, wie z.B. die Optimierung von Transportrouten, die Netzwerkanalyse von Lieferketten oder die Identifizierung von Engpässen.

Structured Query Language (SQL)/ Relational

Anfang 1980er
Sehr hohe
Verbreitung

relational



No not only Structured Query Language (No - SQL)

Mitte 1960er
Nahezu komplett
abgelöst

hierar-
chisch



Ende 1980er
Niedrige
Verbreitung

objekt-
orientiert



Anfang 2000er
mittlere
Verbreitung

document
store

```
data{  
  data1 {  
    "name": "data1",  
  },  
  data2 {  
    "name": "data2",  
  }  
}
```

Mitte 2000er
Relativ niedrige
Verbreitung

graph
store



Mitte 2000er
niedrige bis mittlere
Verbreitung

wide column
stores



Mitte 2010er
niedrige bis mittlere
Verbreitung

key-value
store



```
redis> GET nonexistent
(nil)
redis> SET mykey "Hello"
"OK"
redis> GET mykey
"Hello"
redis>
```

- **In-Memory-Datenbanken:** In-Memory-Datenbanken wie Redis werden in der Logistik verwendet, um schnell auf häufig abgerufene Daten zuzugreifen und die Leistung zu verbessern, insbesondere für Anwendungen, die Echtzeitverarbeitung und schnelle Antwortzeiten erfordern, wie z.B. das Echtzeit-Tracking von Lieferungen oder die Optimierung von Transportrouten.

```
# Fahrzeugstandortdaten aktualisieren
redis-cli SET vehicle1 '{"latitude": 51.5074, "longitude": -0.1278}'
redis-cli SET vehicle2 '{"latitude": 40.7128, "longitude": -74.0060}'

# Fahrzeugstandortdaten abrufen
redis-cli GET vehicle1
redis-cli GET vehicle2

# Fahrzeugstandortdaten löschen
redis-cli DEL vehicle1
```


Link zu redis

1. SQL-Datenbanken
2. No-SQL Datenbanken
 - 2.1 Dokumentenbasierte Datenbanken
 - 2.2 Graphen Datenbanken
 - 2.3 Key Value Datenbanken
3. Beispielhafte Anwendung verschiedener Datenbankarten in der Logistik

Zentrale Herausforderung: Unterschiedliche Datentypen

- Strukturierte Daten: Bestellungen, Kunden, Rechnungen
- Semi-strukturierte Daten: Tracking-Events
- Unstrukturierte Daten: PDFs, Bilder
- Graphstrukturen: Lieferketten, Routen

Konsequenz:

- Eine Datenbank allein reicht nicht aus
- Unterschiedliche Anforderungen:
 - ▶ Konsistenz vs. Performance
 - ▶ Struktur vs. Flexibilität

Lösung: Polyglot Persistence

Zentrale Funktion: Transaktionales Kernsystem

- Speicherung kritischer Geschäftsdaten
- Grundlage für ERP-Systeme
- Garantiert Datenintegrität

Typische Daten

- Orders (OrderID, CustomerID, Status)
- OrderDetails (Produkte, Mengen)
- Lagerbestände

Eigenschaft:

- Single Source of Truth

Warum SQL für Kernsysteme?

- ACID:
 - ▶ Atomicity (alles oder nichts)
 - ▶ Consistency (gültiger Zustand)
 - ▶ Isolation (parallel sicher)
 - ▶ Durability (dauerhaft gespeichert)
- Relationales Modell:
 - ▶ Fremdschlüssel
 - ▶ Normalisierung
- Komplexe Abfragen:
 - ▶ JOINS über mehrere Tabellen

Beispiel: Analyse von Verkäufen pro Kunde

Zentrale Funktion: Event- und Dokumentenspeicher

- Speicherung von Tracking-Daten
- Flexible Datenmodelle
- Aggregation von Events

Typische Struktur

- Ein Dokument pro Lieferung
- Eingebettete Events (Embedding)

Vorteil:

- Alle Informationen an einem Ort

Warum NoSQL?

- Schema-Flexibilität:
 - ▶ unterschiedliche Felder möglich
- JSON-basierte Speicherung:
 - ▶ natürlich für APIs
- Performance:
 - ▶ weniger Joins durch Embedding
- Skalierbarkeit:
 - ▶ horizontale Skalierung

Use Case: Tracking einer Lieferung über mehrere Stationen

Zentrale Funktion: Modellierung von Beziehungen

- Darstellung von Netzwerken
- Fokus auf Knoten und Kanten
- Abfragen über Pfade

Typische Anwendungen

- Lieferketten
- Transportnetzwerke
- Abhängigkeiten zwischen Lieferanten

Warum Graphdatenbanken?

- Beziehungen sind First-Class Citizens
- Keine komplexen JOINS notwendig
- Sehr schnelle Traversierung

Beispiel

- Kürzester Pfad zwischen zwei Städten
- Analyse indirekter Abhängigkeiten

Vorteil gegenüber SQL:

- exponentielle Performance bei Netzwerkabfragen

Zentrale Funktion: Speicherung unstrukturierter Daten

- Speicherung von Dateien als Objekte
- Kein Schema notwendig
- Zugriff über Schlüssel (Key)

Typische Inhalte

- Rechnungen (PDF)
- Lieferscheine
- Bilder / Scans

Warum Object Storage?

- Skalierbarkeit:
 - ▶ Millionen von Dateien möglich
- S3-Kompatibilität:
 - ▶ Industriestandard
- Kosten:
 - ▶ günstiger als klassische DB-Speicherung

Use Case

- Archivierung von Dokumenten

Zentrale Funktion: Echtzeit- und Cache-System

- Speicherung aktueller Zustände
- Beschleunigung von Abfragen
- Entlastung der Datenbanken

Typische Daten

- aktueller Lieferstatus
- Sessions
- API-Caching

Warum Redis?

- RAM-basierte Speicherung → extrem schnell
- TTL:
 - ▶ automatische Löschung
- Key-Value-Modell:
 - ▶ einfache Struktur

Beispiel

- shipment:123 → in_transit

Typische Architektur

- SQL:
 - ▶ Bestellungen und Stammdaten
- MongoDB:
 - ▶ Tracking-Events
- Neo4j:
 - ▶ Routen und Netzwerke
- MinIO:
 - ▶ Dokumente
- Redis:
 - ▶ aktueller Status

Amazon

- SQL + NoSQL + Redis + S3

DHL

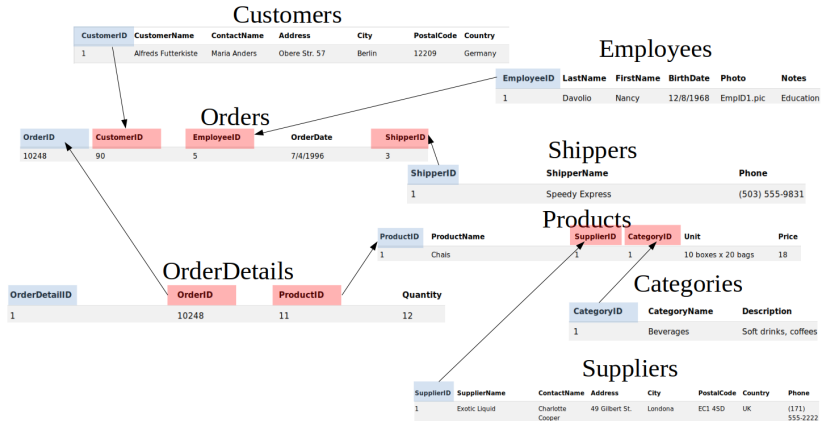
- SQL (ERP)
- Event-Datenbanken für Tracking
- Redis für Echtzeit

Maersk

- Graphdatenbanken für Lieferketten

- Kleine Unternehmen:
 - ▶ meist nur SQL
- KMU:
 - ▶ SQL + Redis + Storage
- Mittelstand:
 - ▶ mehrere Systeme
- Konzerne:
 - ▶ vollständige Polyglot-Architektur

Ursprung der Stammdaten für Polyglot Logistik-Beispiel: W3-schools dataset



Order → Fulfillment → Lieferung

- PostGres/SQL: Customer, Shippers, Order, OrderDetails, Products, Warehouse, Inventory
- MongoDB: docs
- neo4j: neo4j
- redis: keys
- minio: docs